A focused workbook for the next step in your Python basics

# Set Methods

50 focused programming problems

## PRACTICE THESE METHODS

| | | | |
|---|---|---|---|
| **add()** | Add a value | **remove()** | Remove a known value |
| **clear()** | Empty the set | **pop()** | Remove and return one value |
| **union()** | Combine unique values | **intersection()** | Find shared values |

## WHAT YOU WILL PRACTISE

Choose the right set method, reason about unique values, combine sets, safely remove values, and use if/else to make simple decisions.

| 15 | 15 | 15 | 5 |
|---|---|---|---|
| Easy | Medium | Hard | Complex |

## HOW TO USE THIS WORKBOOK

1. Solve each question in VS Code without copying a solution.
2. Run the supplied test case, then change at least one value to test your understanding.
3. Save your code in clearly named .py files and push your work to GitHub.
4. Mark a question complete only when you can explain why the result is correct.

**SCOPE NOTE**

Questions use the set methods listed here, if/else, and your previously studied string/list methods when useful. No loops, dictionaries, or unrelated advanced methods are required. This workbook includes expected results but no full solutions.

# Quick Reference

Read this page first. The questions test the behavior below, including easy-to-miss details.

### add(value)

Adds one value to the existing set. A duplicate value does not create a second copy. The method changes the set and returns None.

```
skills = {"Python"}; skills.add("SQL")
```

### remove(value)

Removes the specified value. It is intended for a value you know is present. Use if value in items before calling remove() when it may be absent.

```
if "SQL" in skills: skills.remove("SQL")
```

### clear()

Removes every value from the existing set. The set itself remains, but it becomes empty.

```
skills.clear()  # skills becomes set()
```

### pop()

Removes and returns one arbitrary value. Do not expect a particular item when a set has multiple values. A one-item set has only one possible value to remove.

```
removed = skills.pop()
```

### union(other)

Returns a new set with all unique values from either set. The original sets are not changed by union().

```
combined = first.union(second)
```

### intersection(other)

Returns a new set containing only values found in both sets. The original sets are not changed by intersection().

```
shared = first.intersection(second)
```

**ALWAYS REMEMBER**

Sets store unique values and do not have a guaranteed display order. Values are case-sensitive: "Python" and "python" are different. Use set() for an empty set; {} creates an empty dictionary. Membership checks use in / not in.

PRACTICE SECTION

# EASY

Build confidence one operation at a time. Focus on exact behavior and simple membership checks.

## E01 Add a workshop guest

FOCUS: ADD()

A workshop already has two registered guests. Add the third guest to the existing set, then print the set.

TEST CASE

```
guests = {"Aarav", "Mira"}
new_guest = "Ishaan"
```

EXPECTED RESULT

**The set contains Aarav, Mira, and Ishaan.**

MY PLAN / NOTES

## E02 Try adding a duplicate ID

FOCUS: ADD()

Add the supplied ID to the set. The ID is already present, so the set should not gain a second copy.

TEST CASE

```
ticket_ids = {12, 18}
new_id = 12
```

EXPECTED RESULT

**The set still contains only 12 and 18.**

MY PLAN / NOTES

## E03 Start with an empty set

FOCUS: SET(), ADD()

Create an empty set correctly, add the label "urgent", and print the result. Remember that {} is not an empty set.

TEST CASE

```
labels = set()
new_label = "urgent"
```

EXPECTED RESULT

**{"urgent"}**

MY PLAN / NOTES

PRACTICE SECTION

# EASY

Build confidence one operation at a time. Focus on exact behavior and simple membership checks.

## E04 Remove an inactive member

FOCUS: REMOVE()

Remove Kabir from the team. Kabir is known to be in the set, so use remove() directly.

TEST CASE

```
team = {"Riya", "Kabir", "Noor"}
```

EXPECTED RESULT

**The set contains Riya and Noor.**

MY PLAN / NOTES

## E05 Remove an old status label

FOCUS: REMOVE()

Remove the status "review" from the labels set. Keep the other status labels unchanged.

TEST CASE

```
labels = {"new", "review", "done"}
```

EXPECTED RESULT

**{"new", "done"}**

MY PLAN / NOTES

## E06 Clear temporary choices

FOCUS: CLEAR()

The choices are no longer needed. Empty the existing set and print it.

TEST CASE

```
choices = {"A", "B", "C"}
```

EXPECTED RESULT

**set()**

MY PLAN / NOTES

PRACTICE SECTION

# EASY

Build confidence one operation at a time. Focus on exact behavior and simple membership checks.

## E07 Pop one colour

FOCUS: POP()

Call pop(), store the removed colour, then print that colour and the remaining set. Do not assume which colour is removed.

TEST CASE

```
colours = {"red", "green", "blue"}
```

EXPECTED RESULT

**The removed value is one of the three colours; the set left behind contains the other two.**

MY PLAN / NOTES

## E08 Combine two book shelves

FOCUS: UNION()

Make a new set containing every unique book category found on either shelf.

TEST CASE

```
shelf_a = {"Python", "SQL"}
shelf_b = {"SQL", "HTML"}
```

EXPECTED RESULT

**{"Python", "SQL", "HTML"}**

MY PLAN / NOTES

## E09 Find shared study days

FOCUS: INTERSECTION()

Find the days that appear in both sets. Store the answer in a new set.

TEST CASE

```
plan_a = {"Mon", "Wed", "Fri"}
plan_b = {"Wed", "Fri", "Sat"}
```

EXPECTED RESULT

**{"Wed", "Fri"}**

MY PLAN / NOTES

PRACTICE SECTION

# EASY

Build confidence one operation at a time. Focus on exact behavior and simple membership checks.

## E10 Check course membership

FOCUS: IN

Print whether "SQL" is present in the supplied course set.

TEST CASE

```
courses = {"Python", "SQL", "HTML"}
```

EXPECTED RESULT

**True**

MY PLAN / NOTES

## E11 Check a blocked access code

FOCUS: IN, IF/ELSE

If the supplied code is in blocked_codes, print "Blocked". Otherwise print "Allowed".

TEST CASE

```
blocked_codes = {"A14", "B07"}
code = "B07"
```

EXPECTED RESULT

**Blocked**

MY PLAN / NOTES

## E12 Notice case sensitivity

FOCUS: IN

Check whether "Python" is in this set. Do not change the case of either value.

TEST CASE

```
skills = {"python", "sql"}
```

EXPECTED RESULT

**False**

MY PLAN / NOTES

PRACTICE SECTION

# EASY

Build confidence one operation at a time. Focus on exact behavior and simple membership checks.

## E13 Keep the source sets unchanged

FOCUS: UNION()

Create and print a combined set. Then print morning again to confirm union() did not change it.

TEST CASE

```
morning = {"tea", "toast"}
evening = {"toast", "fruit"}
```

EXPECTED RESULT

**Combined: {"tea", "toast", "fruit"}; morning remains {"tea", "toast"}.**

MY PLAN / NOTES

## E14 Find no shared supplies

FOCUS: INTERSECTION()

Find the supplies present in both sets. The answer should be an empty set because there are no shared values.

TEST CASE

```
kitchen = {"plate", "cup"}
desk = {"pen", "book"}
```

EXPECTED RESULT

**set()**

MY PLAN / NOTES

## E15 Remove only when present

FOCUS: IN, REMOVE(), IF/ELSE

Try to remove "eraser" safely. If it is absent, print "Item not in basket" and do not call remove() on it.

TEST CASE

```
basket = {"pen", "pencil"}
item = "eraser"
```

EXPECTED RESULT

**Item not in basket; basket remains {"pen", "pencil"}.**

MY PLAN / NOTES

PRACTICE SECTION

# MEDIUM

Combine set methods with if/else and check what changes - and what stays unchanged.

## M01 Create a club sign-up report

FOCUS: UNION(), INTERSECTION()

Print both the full roster across the two clubs and the people registered for both clubs.

TEST CASE

```
robotics = {"Asha", "Dev", "Mina"}
coding = {"Dev", "Jay", "Mina"}
```

EXPECTED RESULT

**Combined: Asha, Dev, Mina, Jay. Shared: Dev, Mina. Set order may vary.**

MY PLAN / NOTES

## M02 Include a late attendee

FOCUS: ADD(), UNION()

Add Neel to confirmed, then combine confirmed with walk_ins. Print the combined set.

TEST CASE

```
confirmed = {"Ravi", "Tia"}
walk_ins = {"Uma", "Ravi"}
late_guest = "Neel"
```

EXPECTED RESULT

**The combined set contains Ravi, Tia, Uma, and Neel.**

MY PLAN / NOTES

## M03 Remove a device safely

FOCUS: IN, REMOVE(), IF/ELSE

If mouse is present, remove it and print "Removed". Otherwise print "Not listed". Print the final set too.

TEST CASE

```
devices = {"keyboard", "mouse", "monitor"}
target = "mouse"
```

EXPECTED RESULT

**Removed; remaining set contains keyboard and monitor.**

MY PLAN / NOTES

PRACTICE SECTION

# MEDIUM

Combine set methods with if/else and check what changes - and what stays unchanged.

## M04 Check whether both clubs offer Robotics

FOCUS: INTERSECTION(), IF/ELSE

Find the shared club activities. Use if/else to print "Both groups offer Robotics" only if Robotics is in the intersection.

TEST CASE

```
group_a = {"Chess", "Robotics"}
group_b = {"Robotics", "Music"}
```

EXPECTED RESULT

**Both groups offer Robotics**

MY PLAN / NOTES

## M05 Edit the combined result only

FOCUS: UNION(), ADD()

Create combined from A and B, add D to combined, then print all three sets. A and B must remain unchanged.

TEST CASE

```
A = {"A", "B"}
B = {"B", "C"}
```

EXPECTED RESULT

**combined contains A, B, C, D; original A and B are unchanged.**

MY PLAN / NOTES

## M06 Reset and rebuild today's tags

FOCUS: CLEAR(), ADD()

Clear the old tags, then add "new" and "review". Print the final set.

TEST CASE

```
today_tags = {"draft", "old"}
```

EXPECTED RESULT

**{"new", "review"}**

MY PLAN / NOTES

PRACTICE SECTION

# MEDIUM

Combine set methods with if/else and check what changes - and what stays unchanged.

## M07 Check the item returned by pop()

FOCUS: POP(), IN, IF/ELSE

Store the value returned by pop(). Use if/else to check whether that value is still in the remaining set, then print the remaining set.

TEST CASE

```
letters = {"x", "y", "z"}
```

EXPECTED RESULT

**Prints "Removed"; the set contains the two values not popped.**

MY PLAN / NOTES

## M08 Remove two optional labels safely

FOCUS: IN, REMOVE(), IF/ELSE

Try to remove "review" and then "urgent". For each label, print either "Removed: label" or "Missing: label" without causing an error.

TEST CASE

```
labels = {"new", "review"}
```

EXPECTED RESULT

**Removed: review; Missing: urgent; final set is {"new"}.**

MY PLAN / NOTES

## M09 Check the combined book shelves

FOCUS: UNION(), IN, IF/ELSE

Combine both shelves, then print "Ready to issue" if "Python Basics" is in the combined set. Otherwise print "Unavailable".

TEST CASE

```
shelf_a = {"Python Basics", "SQL Guide"}
shelf_b = {"HTML Guide"}
```

EXPECTED RESULT

**Ready to issue**

MY PLAN / NOTES

PRACTICE SECTION

# MEDIUM

Combine set methods with if/else and check what changes - and what stays unchanged.

## M10 Keep a saved intersection

FOCUS: INTERSECTION(), REMOVE()

Create shared from group_a and group_b. Then remove Tara from group_a and print both sets. Tara should still appear in shared.

TEST CASE

```
group_a = {"Mira", "Tara", "Aman"}
group_b = {"Tara", "Aman"}
```

EXPECTED RESULT

**shared still contains Tara and Aman; group_a contains Mira and Aman.**

MY PLAN / NOTES

## M11 Compare names with different casing

FOCUS: UNION(), INTERSECTION()

Print the union and intersection. Explain in a comment why Anaya and anaya are not treated as the same value.

TEST CASE

```
first = {"Anaya", "Rahul"}
second = {"anaya", "Rahul"}
```

EXPECTED RESULT

**Union contains Anaya, anaya, and Rahul; intersection contains Rahul.**

MY PLAN / NOTES

## M12 Clean a label before saving it

FOCUS: REPLACE(), CAPITALIZE(), ADD()

Replace the hyphen with a space, apply capitalize(), and add the final label to tags. Store each returned string.

TEST CASE

```
label = "python-basics"
tags = {"coding"}
```

EXPECTED RESULT

**tags contains "coding" and "Python basics".**

MY PLAN / NOTES

PRACTICE SECTION

# MEDIUM

Combine set methods with if/else and check what changes - and what stays unchanged.

## M13 Remove a permission from a combined set

FOCUS: UNION(), IN, REMOVE()

Combine both permission sets. If "write" is present in the combined result, remove it. Print the result and confirm access_b is unchanged.

TEST CASE

```
access_a = {"read", "print"}
access_b = {"read", "write"}
```

EXPECTED RESULT

**Combined result contains read and print; access_b still contains read and write.**

MY PLAN / NOTES

## M14 Find a missing ticket

FOCUS: UNION(), IN, IF/ELSE

Combine the ticket sets and check for A-3. Print "Ticket found" if present; otherwise print "Ask the organizer".

TEST CASE

```
first = {"A-1", "A-2"}
second = {"B-1"}
requested = "A-3"
```

EXPECTED RESULT

**Ask the organizer**

MY PLAN / NOTES

## M15 Pop a one-item set

FOCUS: POP(), IN, IF/ELSE

Pop the only case ID and print it. Then use if/else to report whether it is still pending, and print the remaining set.

TEST CASE

```
pending = {"CASE-17"}
```

EXPECTED RESULT

**Prints CASE-17, then "No longer pending", then set().**

MY PLAN / NOTES

PRACTICE SECTION

# HARD

Reason about returned sets, mutation, safe removal, case sensitivity, and pop() behavior.

H01

## Update a team before reporting overlap

FOCUS: ADD(), UNION(), INTERSECTION()

Add Mira to team_a. Then print the combined team and the names shared by both teams.

TEST CASE

```
team_a = {"Asha", "Dev"}
team_b = {"Dev", "Karan"}
```

EXPECTED RESULT

**Combined contains Asha, Dev, Mira, Karan; shared contains Dev.**

MY PLAN / NOTES

H02

## Remove from a source after intersecting

FOCUS: INTERSECTION(), REMOVE()

Create shared from seniors and mentors, then remove Dev from seniors. Print both. The saved shared set should still contain Dev.

TEST CASE

```
seniors = {"Asha", "Dev", "Mira"}
mentors = {"Dev", "Mira"}
```

EXPECTED RESULT

**shared contains Dev and Mira; seniors contains Asha and Mira.**

MY PLAN / NOTES

H03

## Build a cleaned visitor roster

FOCUS: UNION(), IN, REMOVE()

Combine both attendance sets. If "visitor" is present, remove it; if "staff-2" is present, remove it too. Print the cleaned roster.

TEST CASE

```
morning = {"visitor", "staff-1"}
evening = {"staff-1", "staff-2"}
```

EXPECTED RESULT

**{"staff-1"}**

MY PLAN / NOTES

PRACTICE SECTION

# HARD

Reason about returned sets, mutation, safe removal, case sensitivity, and pop() behavior.

H04

## Clear a source without clearing its intersection

FOCUS: INTERSECTION(), CLEAR()

Create common from android and ios, then clear android. Print common, android, and ios to see which set was emptied.

TEST CASE

```
android = {"Asha", "Kabir"}
ios = {"Kabir", "Mira"}
```

EXPECTED RESULT

**common contains Kabir; android is set(); ios is unchanged.**

MY PLAN / NOTES

H05

## Do not print add() as if it were a set

FOCUS: ADD()

Run the two print statements shown in your solution: first print the value returned by add(), then print skills. Explain the difference in a comment.

TEST CASE

```
skills = {"Python"}
result = skills.add("SQL")
```

EXPECTED RESULT

**First print is None; the second shows a set containing Python and SQL.**

MY PLAN / NOTES

H06

## Add a repeated visitor ID

FOCUS: ADD()

Add V-12 again, then add V-13. Print visitors. Each distinct ID should appear only once.

TEST CASE

```
visitors = {"V-11", "V-12"}
```

EXPECTED RESULT

**The set contains V-11, V-12, and V-13.**

MY PLAN / NOTES

PRACTICE SECTION

# HARD

Reason about returned sets, mutation, safe removal, case sensitivity, and pop() behavior.

## H07 Recover the value popped from a set

FOCUS: POP(), ADD()

Pop one value, store it, and then add that same value back. Print the set. It should contain all the original values regardless of which one pop() chose.

TEST CASE

```
colours = {"amber", "blue", "green"}
```

EXPECTED RESULT

**The final set contains amber, blue, and green.**

MY PLAN / NOTES

## H08 Find exact shared app names

FOCUS: INTERSECTION()

Print the intersection. Add a comment explaining why only exact, case-matching names count as shared.

TEST CASE

```
apps_a = {"Zoom", "Teams", "slack"}
apps_b = {"zoom", "Teams", "Slack"}
```

EXPECTED RESULT

**{"Teams"}**

MY PLAN / NOTES

## H09 Clear a combined set, not its sources

FOCUS: UNION(), CLEAR()

Create combined using union(), clear combined, and print all three sets. Do not clear A or B.

TEST CASE

```
A = {"draft", "review"}
B = {"review", "done"}
```

EXPECTED RESULT

**combined is set(); A and B retain their original values.**

MY PLAN / NOTES

PRACTICE SECTION

# HARD

Reason about returned sets, mutation, safe removal, case sensitivity, and pop() behavior.

H10

## Handle an ID that is not in the roster

FOCUS: UNION(), IN, REMOVE(), IF/ELSE

Create all_ids using union(). Remove C-04 only if present; otherwise print "No matching ID". Print all_ids at the end.

TEST CASE

```
A = {"C-01", "C-02"}
B = {"C-02", "C-03"}
target = "C-04"
```

EXPECTED RESULT

**No matching ID; all_ids still contains C-01, C-02, and C-03.**

MY PLAN / NOTES

H11

## Add a manual override to shared access

FOCUS: INTERSECTION(), ADD()

Find the shared members, add "Admin override" to that result, then print the result and both original sets.

TEST CASE

```
registered = {"Asha", "Dev", "Mira"}
paid = {"Dev", "Mira", "Karan"}
```

EXPECTED RESULT

**Result contains Dev, Mira, Admin override; registered and paid are unchanged.**

MY PLAN / NOTES

H12

## Clear a combined result safely

FOCUS: UNION(), CLEAR()

Create latest by combining old_tags with {"green"}. Clear latest and print both sets. Explain why old_tags is not cleared.

TEST CASE

```
old_tags = {"red", "blue"}
```

EXPECTED RESULT

**latest is set(); old_tags still contains red and blue.**

MY PLAN / NOTES

PRACTICE SECTION

# HARD

Reason about returned sets, mutation, safe removal, case sensitivity, and pop() behavior.

H13

## Block a permission after combining roles

FOCUS: UNION(), IN, REMOVE(), IF/ELSE

Combine the permissions. Remove "delete" if present, then use if/else to report whether delete remains in the cleaned set.

TEST CASE

```
day_role = {"read", "write"}
night_role = {"read", "delete"}
```

EXPECTED RESULT

**Prints "Delete blocked"; cleaned set contains read and write.**

MY PLAN / NOTES

H14

## Prove union and intersection results are independent

FOCUS: UNION(), INTERSECTION(), ADD(), CLEAR()

Create joined and shared from A and B. Add q to joined and clear shared. Print all four sets.

TEST CASE

```
A = {"x", "y"}
B = {"y", "z"}
```

EXPECTED RESULT

**A and B are unchanged; joined contains x, y, z, q; shared is set().**

MY PLAN / NOTES

H15

## Reject a differently cased token

FOCUS: IN, REMOVE(), IF/ELSE

Try to remove TOKEN from tokens only if it is present. Otherwise print "Exact token not found". Keep Token and token unchanged.

TEST CASE

```
tokens = {"Token", "token"}
target = "TOKEN"
```

EXPECTED RESULT

**Exact token not found; both original values remain.**

MY PLAN / NOTES

PRACTICE SECTION

# COMPLEX / REAL-WORLD

Small practical tasks that combine several studied methods without introducing loops or advanced syntax.

## C01 Finalize a shared event roster

FOCUS: UNION(), INTERSECTION(), ADD(), REMOVE()

Combine coding and design attendees, add Nia, then remove Kabir only if present. Also print the original shared attendees using intersection(). Show the cleaned roster and the shared set.

TEST CASE

```
coding = {"Asha", "Dev", "Mira"}
design = {"Dev", "Mira", "Kabir"}
```

EXPECTED RESULT

**Cleaned roster contains Asha, Dev, Mira, Nia. Shared set contains Dev and Mira. Source sets stay unchanged.**

MY PLAN / NOTES

## C02 Clean permissions without editing the source roles

FOCUS: UNION(), IN, REMOVE(), IF/ELSE

Combine the two roles into access. Remove export if present. Then use if/else to report whether write remains. Print access and both source roles.

TEST CASE

```
team_a = {"read", "write", "export"}
team_b = {"read", "comment"}
```

EXPECTED RESULT

**access contains read, write, comment; prints "Write access remains"; team_a still contains export.**

MY PLAN / NOTES

## C03 Audit a packed workshop kit

FOCUS: INTERSECTION(), IN, IF/ELSE

Find which packed items are also required using intersection(). Then check whether adapter is packed and print either "Adapter ready" or "Adapter missing".

TEST CASE

```
packed = {"charger", "mouse", "keyboard"}
required = {"charger", "mouse", "adapter"}
```

EXPECTED RESULT

**Shared items: charger and mouse. Prints "Adapter missing".**

MY PLAN / NOTES

PRACTICE SECTION

# COMPLEX / REAL-WORLD

Small practical tasks that combine several studied methods without introducing loops or advanced syntax.

## C04 Normalize and register a topic tag

FOCUS: REPLACE(), CAPITALIZE(), ADD(), IN, IF/ELSE

Fix the label by replacing PYTHON with Python, replacing the hyphen with a space, then using capitalize(). If the final label already exists in tags, print "Already registered"; otherwise add it. Store each returned string.

TEST CASE

```
label = "PYTHON-basics"
tags = {"coding", "Python basics"}
```

EXPECTED RESULT

**Final label is Python basics; prints "Already registered"; tags stays unique.**

MY PLAN / NOTES

## C05 Prepare the final paid workshop roster

FOCUS: UNION(), ADD(), REMOVE(), INTERSECTION(), IF/ELSE

Combine morning and afternoon sign-ups, add Tara, remove Mira if present, and intersect the result with paid. Finally, use if/else to report whether Dev is eligible. Print the eligible set and original sets.

TEST CASE

```
morning = {"Asha", "Dev", "Mira"}
afternoon = {"Dev", "Kabir"}
paid = {"Dev", "Mira", "Kabir"}
```

EXPECTED RESULT

**Eligible set contains Dev and Kabir; prints "Dev is eligible". Original sets are unchanged.**

MY PLAN / NOTES

# Progress Tracker

Track independent practice, not just whether the code ran once.

## Easy

| | | | | |
|---|---|---|---|---|
| ☐ E01 | ☐ E02 | ☐ E03 | ☐ E04 | ☐ E05 |
| ☐ E06 | ☐ E07 | ☐ E08 | ☐ E09 | ☐ E10 |
| ☐ E11 | ☐ E12 | ☐ E13 | ☐ E14 | ☐ E15 |

## Medium

| | | | | |
|---|---|---|---|---|
| ☐ M01 | ☐ M02 | ☐ M03 | ☐ M04 | ☐ M05 |
| ☐ M06 | ☐ M07 | ☐ M08 | ☐ M09 | ☐ M10 |
| ☐ M11 | ☐ M12 | ☐ M13 | ☐ M14 | ☐ M15 |

## Hard

| | | | | |
|---|---|---|---|---|
| ☐ H01 | ☐ H02 | ☐ H03 | ☐ H04 | ☐ H05 |
| ☐ H06 | ☐ H07 | ☐ H08 | ☐ H09 | ☐ H10 |
| ☐ H11 | ☐ H12 | ☐ H13 | ☐ H14 | ☐ H15 |

## Complex / real-world

| | | | | |
|---|---|---|---|---|
| ☐ C01 | ☐ C02 | ☐ C03 | ☐ C04 | ☐ C05 |

**A QUESTION IS COMPLETE WHEN YOU CAN**

1. Explain what the method changes or returns.
2. Match the supplied expected result and test one changed input.
3. Write the solution again without looking at your previous code.